# Python OOP Interview Explorer

An interactive dashboard to explore 30 essential Object-Oriented Programming questions for interview preparation.

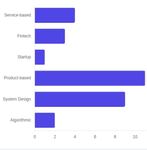
Filter by OOP Concept: All Concepts | Filter by Company Type: All Company Types | Search Keywords: e.g., 'Stack', 'Bank', 'Shape'... | Reset Filters

## Question Bank Overview

### Questions by OOP Concept


Click a segment to filter questions below.

### Focus by Company Type




## Practice Questions

30 Questions

Basic Class

### 1. Car Class: Basic Information Display

**Company Focus:** TCS, Infosys, Wipro, General IT Service Companies

Create a `Car` class with attributes `make`, `model`, and `year`. Implement a method, `display_info()`, that prints all attributes in a readable format.

Sample Input:

```
c = Car("Tata", "Nexon", 2023)
c.display_info()
```

Expected Output:

```
Car: 2023 Tata Nexon
```

Encapsulation

### 2. Employee Bonus Calculation

**Company Focus:** Capgemini, HCL, Mid-tier IT

Implement an `Employee` class with **private attributes** `__name`, `__age`, and `__salary`. Add a public method, `calculate_bonus()`, that returns a 10% bonus on the salary, testing access control.

Sample Input:

```
e = Employee("Priya", 28, 60000)
print(f"Bonus: {e.calculate_bonus()}")
```

Expected Output:

```
Bonus: 6000.0
```

Encapsulation

### 3. BankAccount Transaction Control

**Company Focus:** Accenture, Cognizant, Fintech

Create a `BankAccount` class with a private `__balance`. Implement `deposit()` and `withdraw()` methods. The `withdraw()` method must check for sufficient funds and prevent the balance from going negative.

Sample Input:

```
acc = BankAccount(1000)
acc.withdraw(1200)
acc.deposit(500)
```

Expected Output:

```
Withdrawal failed: Insufficient f
Deposit successful. Balance: 1500
```

Basic Class

### 4. Student Grade Analysis

**Company Focus:** Tech Mahindra, Startups

Implement a `Student` class with attributes `name` and a list of `grades`. Add methods to `calculate_average()`, `find_highest_grade()`, and `find_lowest_grade()`.

Sample Input:

```
s = Student("Amar", [88, 92, 75,
print(f"Average: {s.calculate_ave
```

Expected Output:

```
Average: 87.5
```

Basic Class

### 5. Complex Number Addition

**Company Focus:** General Product/Support Roles

Create a class `Complex` to represent complex numbers (`$a + b$`). Implement a method, `add(other)`, that returns a new `Complex` object representing the sum of two complex numbers.

Sample Input:

```
c1 = Complex(3, 2)
c2 = Complex(1, 7)
c3 = c1.add(c2)
c3.display()
```

Expected Output:

```
4 + 9i
```

Composition

### 6. Library Management

**Company Focus:** L&T Infotech, System Integrators

Design a `Library` class that manages a collection of `Book` objects (Composition). Implement methods to `add_book()` and `find_book(title)`.

Sample Input:

```
lib = Library()
lib.add_book("OOP Python")
lib.find_book("OOP Python")
```

Expected Output:

```
Book found: OOP Python
```

Polymorphism

### 7. Animal Sounds

**Company Focus:** General Service-based Companies

Create a parent class `Animal` with a generic method `make_sound()`. Create subclasses `Dog`, `Cat`, and `Cow` that **override** `make_sound()` with their specific sounds, demonstrating Polymorphism.

Sample Input:

```
animals = [Dog(), Cat(), Cow()]
for a in animals: a.make_sound()
```

Expected Output:

```
Woof! Woof!
Meow...
Moo!
```

Abstraction

### 8. Shape Area Calculation

**Company Focus:** Mid-to-Large Product Companies

Define an **abstract base class** `Shape` with an abstract method `area()`. Create concrete subclasses `Circle` (needs radius) and `Rectangle` (needs width, height) that correctly implement the `area()` method.

Sample Input:

```
r = Rectangle(10, 5)
r.area()
```

Expected Output:

```
50
```

Encapsulation

### 9. Savings vs Checking Account

**Company Focus:** FinTech, Banking Systems

Inherit from the `BankAccount` class (Q3). Create `SavingsAccount` which overrides the `withdraw()` method to enforce a minimum balance of $500.

Sample Input:

```
sa = SavingsAccount(600)
sa.withdraw(150)
```

Expected Output:

```
Withdrawal failed: Minimum balanc
```

Inheritance

### 10. Vehicle Multilevel Hierarchy

**Company Focus:** Automotive Tech, Embedded Systems

Create a multilevel hierarchy: `Vehicle` (parent) → `FourWheeler` (child) → `Car` (grandchild). Demonstrate inheritance by defining a method in `Vehicle` that is accessible by `Car`.

Sample Input:

```
c = Car("Sedan")
c.drive()
```

Expected Output:

```
The vehicle is moving.
```

Abstraction

### 11. Payment Gateway

**Company Focus:** E-commerce, FinTech

Define an abstract class `PaymentGateway` with an abstract method `process_payment()`. Create concrete classes `CreditCardPayment` and `UPIPayment` that implement this method differently.

Sample Input:

```
p = UPIPayment()
p.process_payment(200)
```

Expected Output:

```
UPI payment of $200 processed.
```

Inheritance

### 12. Geometry Perimeter

**Company Focus:** General Tech, Geometry Apps

Create a class hierarchy for calculating the perimeter. Parent: `Shape`. Children: `Rectangle` and `Triangle`. Ensure each child correctly calculates its perimeter.

Sample Input:

```
t = Triangle(3, 4, 5)
print(t.perimeter())
```

Expected Output:

```
12
```

Inheritance

### 13. Employee Hierarchy (Manager Role)

**Company Focus:** HR Software, Large Corporations

Create classes `Employee` and `Manager` (inherits from `Employee`). `Manager` should have an additional `manage_team()` method and receive a higher base salary upon initialization (by calling `super().__init__`).

Sample Input:

```
m = Manager("Karan", 120000)
m.manage_team()
```

Expected Output:

```
Karan is managing the team.
```

Inheritance

### 14. Utility Bill (Hierarchical Inheritance)

**Company Focus:** Utilities/Energy Tech

Create a parent class `Bill`. Create two children: `ElectricityBill` and `WaterBill`. Both should calculate the total based on usage, but use different rate methods, demonstrating Hierarchical Inheritance.

Sample Input:

```
w = WaterBill(50)
print(f"Total: {w.calculate_total
```

Expected Output:

```
Total: 250.0
```

Polymorphism

### 15. Media Player (Duck Typing)

**Company Focus:** Media/Streaming Services

Create classes `AudioFile` and `VideoFile`. Both must have a `play()` method. Write a standalone function `start_playback(media)` that accepts both types of objects, demonstrating **Duck Typing** (if it walks like a duck...).

Sample Input:

```
v = VideoFile("Movie.mp4")
start_playback(v)
```

Expected Output:

```
Playing video file: Movie.mp4
```

Data Structures

### 16. Stack Class (LIFO)

**Company Focus:** Algorithmic Rounds, General Coding Assessment

Implement a `Stack` class (Last-In, First-Out) using a Python list internally. It must support the operations `push`, `pop`, `peek` (view top element), and `is_empty`. Handle the "pop from empty stack" exception.

Sample Input:

```
s = Stack()
s.push(10)
s.pop()
s.pop()
```

Expected Output:

```
10
Error: Stack is empty.
```

Data Structures

### 17. Queue Class (FIFO)

**Company Focus:** Algorithmic Rounds, General Coding Assessment

Implement a `Queue` class (First-In, First-Out) with methods `enqueue`, `dequeue`, and `size`.

Sample Input:

```
q = Queue()
q.enqueue("Task1")
q.enqueue("Task2")
print(q.dequeue())
```

Expected Output:

```
Task1
```

Data Structures

### 18. Single Linked List Operations

**Company Focus:** Data Structure Specialists, Product Companies

Implement a basic `Node` class and a `LinkedList` class. Implement methods to `insert_at_beginning(data)` and `display` the list elements.

Sample Input:

```
ll = LinkedList()
ll.insert_at_beginning(5)
ll.insert_at_beginning(10)
ll.display()
```

Expected Output:

```
[10 -> 5 -> None]
```

Composition

### 19. Custom Dictionary/Config Store

**Company Focus:** System/Backend Development

Create a class `ConfigStore` that wraps a Python dictionary. Implement methods `get_value(key)`, `set_value(key, value)`, and a method `load_from_db()` (simulated print).

Sample Input:

```
cs = ConfigStore()
cs.set_value("mode", "prod")
print(cs.get_value("mode"))
```

Expected Output:

```
prod
```

Data Structures

### 20. Matrix Addition

**Company Focus:** Data Science, Engineering

Implement a `Matrix` class. The constructor takes a nested list. Implement a method `add(other_matrix)` that returns a new `Matrix` object representing the sum.

Sample Input:

```
m1 = Matrix([[1, 2], [3, 4]])
m2 = Matrix([[5, 6], [7, 8]])
m3 = m1.add(m2)
m3.display()
```

Expected Output:

```
[[6, 8], [10, 12]]
```

Data Structures

### 21. Circular Queue

**Company Focus:** System Design, Low-level Programming

Implement a `CircularQueue` with a fixed size using a list and front/rear pointers. Implement `enqueue` and `dequeue` with wrap-around logic.

Sample Input:

```
cq = CircularQueue(3)
cq.enqueue(1)
cq.enqueue(2)
cq.dequeue()
```

Expected Output:

```
1
```

Data Structures

### 22. Priority Queue Concept

**Company Focus:** Software Architecture, Operating Systems

Define a class `TaskScheduler`. Implement a method `add_task(priority, name)` where tasks are stored based on their priority (higher number = higher priority). Implement a method `get_next_task()` that retrieves the highest priority task.

Sample Input:

```
ts = TaskScheduler()
ts.add_task(3, "High")
ts.add_task(1, "Low")
print(ts.get_next_task())
```

Expected Output:

```
Task: High (Priority: 3)
```

Composition

### 23. Simple File System

**Company Focus:** OS Development, Cloud Systems

Define a `Folder` class and a `File` class. A `Folder` contains a list of `File` objects (Composition). Implement a method in `Folder` to calculate the total size of files inside it.

Sample Input:

```
f1 = File("data.txt", 50)
d1 = Folder("Docs", [f1])
print(d1.total_size())
```

Expected Output:

```
50
```

Advanced

### 24. Time Operator Overloading

**Company Focus:** Python Specialist, Tooling

Create a `Time` class with attributes `hours`, `minutes`, and `seconds`. Implement the special method `__add__` to use the native `+` operator for adding two `Time` objects, correctly handling rollovers (e.g., 60 seconds becomes 1 minute).

Sample Input:

```
t1 = Time(0, 30, 45)
t2 = Time(0, 45, 20)
t3 = t1 + t2
t3.display()
```

Expected Output:

```
Time: 1 hour, 16 minutes, 5 secon
```

Basic Class

### 25. Point Distance Calculation

**Company Focus:** Gaming, Graphics, Math Libraries

Implement a `Point` class with `x` and `y` attributes. Add a method `distance_to(other_point)` to calculate the Euclidean distance between two points.

Sample Input:

```
p1 = Point(1, 1)
p2 = Point(4, 5)
print(f"Distance: {p1.distance_to
```

Expected Output:

```
Distance: 5.0
```

Advanced

### 26. Email Utility (Class Method)

**Company Focus:** General Utilities, Networking

Create an `Email` class. Implement a **class method** `create_draft(recipient, message)` which initializes the object with a *default* sender* (e.g., `default_sender@cc.com`) defined within the class.

Sample Input:

```
e = Email.create_draft("hr@corp.c
print(e.sender)
```

Expected Output:

```
default_sender@cc.com
```

Advanced

### 27. Temperature Property (Getter/Setter)

**Company Focus:** IoT, Data Processing

Implement a `Temperature` class. Use the **`@property` decorator** for `celsius` and **`@celsius.setter`** to ensure that when `celsius` is set, the corresponding `fahrenheit` attribute is automatically updated.

Sample Input:

```
t = Temperature(25)
t.celsius = 0
print(t.fahrenheit)
```

Expected Output:

```
32.0
```

Advanced

### 28. Singleton Pattern Implementation

**Company Focus:** Software Architecture, System Design

Implement a `Logger` class using the **Singleton** design pattern, ensuring that only one instance of the logger can ever be created.

Sample Input:

```
l1 = Logger.get_instance()
l2 = Logger.get_instance()
print(l1 is l2)
```

Expected Output:

```
True
```

Advanced

### 29. Custom Iterator

**Company Focus:** Python Specialist, Frameworks

Create a class `EvenNumbers` that acts as a custom **iterator** and **iterable** (implementing `__iter__` and `__next__`) to yield even numbers up to a specified limit.

Sample Input:

```
g = EvenNumbers(6)
print(list(e))
```

Expected Output:

```
[0, 2, 4, 6]
```

Advanced

### 30. Static Method Utility

**Company Focus:** Testing, Data Preprocessing

Create a class `TextFormatter`. Implement a **static method** `clean_text(text)` that removes leading/trailing spaces and converts the text to title case. No instance of `TextFormatter` should be needed.

Sample Input:

```
print(TextFormatter.clean_text("
```

Expected Output:

```
Hello World
```

## About the Creator


Amar Panchal

Technical Trainer & Co-founder of Career Credentials

17 years of expertise in coding and placement training for freshers.

Connect with Amar Panchal